

Video Transcoding Platform By React & Spring Boot:

Objectives:

1. **Simplified Video Upload:** Create a user-friendly platform where users can upload video files through a simple drag-and-drop interface, without requiring technical knowledge of video formats or codecs.
2. **Multi-Resolution Transcoding:** Automatically convert uploaded videos into multiple resolutions (1080p, 720p, and 480p) so that users can choose the quality best suited to their internet speed and device capability.
3. **Cloud-Based Storage and Instant Playback:** Store original and processed videos on cloud storage and provide publicly accessible streaming URLs, enabling instant video preview directly in the browser.
4. **Scalable Backend Architecture:** Build a robust and modular backend using Java Spring Boot, following industry-standard RESTful API design principles, to ensure the system can be extended with additional features in the future.

Deliverables:

1. **Video Upload Functionality:**
 1. Allow users to upload video files of any size through a drag-and-drop interface.
 2. Display real-time upload progress to the user.
2. **Video Storage Management:**
 1. Save the original uploaded video temporarily on the server's local storage.
 2. Upload the original video to cloud storage for permanent, publicly accessible hosting.
3. **Video Metadata Management:**
 1. Store video details (original filename, unique ID, file size, storage path, cloud URL) in a MongoDB database.
 2. Retrieve and display this metadata on the preview page.
4. **Multi-Resolution Video Transcoding:**
 1. Use FFmpeg to compress and convert the original video into 1080p, 720p, and 480p resolutions.
 2. Perform transcoding on-demand, only when a specific quality is requested by the user, to optimize server resource usage.
5. **Video Preview (Show) Feature:**
 1. Allow users to preview any transcoded resolution directly in the browser before downloading.
 2. Upload the transcoded version to cloud storage and stream it via a public URL.
6. **Video Download Feature:**
 1. Allow users to download the transcoded video in their chosen resolution with a single click.
 2. Stream the compressed file directly from server storage to the user's browser.
7. **RESTful API Backend:**
 1. Build all backend functionality using Spring Boot, exposing clean REST API endpoints for upload, preview, and download operations.
 2. Ensure smooth communication between the React frontend and the Spring Boot backend using CORS configuration.

Requirements:

- **Java & Spring Boot:** Build the backend REST APIs using Spring Boot framework.
- **Maven:** Manage backend project dependencies and build lifecycle.

- **MongoDB:** Set up a MongoDB database to store video metadata.
- **React.js:** Develop the frontend user interface using React.js.
- **React Router:** Implement client-side routing between the upload page and preview page.
- **Axios:** Handle HTTP communication between frontend and backend.
- **TailwindCSS:** Style the frontend components.
- **FFmpeg:** Perform video compression and resolution conversion.
- **Cloudinary / AWS S3:** Store and serve video files via public URLs for streaming and playback.

Advanced Features:

1. **Progress Tracking for Transcoding:**
 1. Display real-time transcoding progress to the user instead of a static loading state.
 2. Implement this using WebSockets or polling mechanisms.
2. **Video Thumbnail Generation:**
 1. Automatically generate a thumbnail image from the uploaded video for preview purposes.
3. **AWS Cloud Integration:**
 1. Deploy the backend on AWS EC2 for public accessibility.
 2. Optionally use AWS services for scalable video storage and delivery.
4. **Adaptive Streaming Support:**
 1. Implement HLS (HTTP Live Streaming) for smoother playback based on the user's network speed.
5. **User Authentication:**
 1. Allow users to create accounts and maintain a history of their uploaded and transcoded videos.
6. **Analytics Dashboard:**
 1. Track and display statistics such as number of views, downloads, and most-used resolutions per video.